RoboCupJunior Rescue (Line) 2026
Team Description Paper
Pharaoh

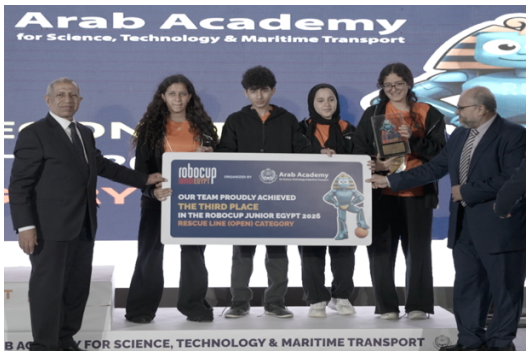
Abstract

The Egypt Pharaoh team has developed a highly autonomous, robust rescue robot utilizing a distributed computing architecture designed specifically to tackle the complexities of the RoboCup Junior Rescue Line challenge. Our robot's core processing is handled by a Raspberry Pi 5 running Python and OpenCV for environmental perception, coupled with a Google Coral USB Accelerator running a custom-trained YOLOv8 AI model for real-time victim detection. Low-level hardware actuation is synchronously controlled by an Arduino Nano via a custom C++ library (RCJBot). Mechanically, the robot features a custom-designed chassis with high ground clearance for speed bumps, REV DUO motion wheels for superior traction, and a rapid-deployment dual-micro-servo victim retrieval mechanism. To ensure absolute electrical stability, we engineered a custom Printed Circuit Board (PCB) using Altium Designer that isolates power delivery between the drive systems and the logic boards. This paper details our progressive engineering methodology, system integration, and rigorous testing protocols that led to our qualification for the World Finals.

1. Introduction

a. Team members

- **Maya Ragab (Main Programmer):** Led the software development process, designing the core state machine, integrating the dual-camera vision system, and training the YOLOv8 victim detection model.
- **Youssef Mohamed (Main Mechanical Designer):** Led the mechanical design using SolidWorks, focusing on structural integrity, high ground clearance, and the rapid prototyping of 3D-printed sub-modules like the battery holder and victim mechanism.
- **Ruba Ahmed (Electrical & Custom PCB Designer):** Designed the electrical system and custom Altium PCB layouts, successfully reducing internal wiring by 60% and solving critical power distribution challenges.
- **Diala Ragab (Second Lead Programmer):** Contributed to the development of the custom C++ library, assisted with UART communication debugging, and optimized the PID line-following algorithms.



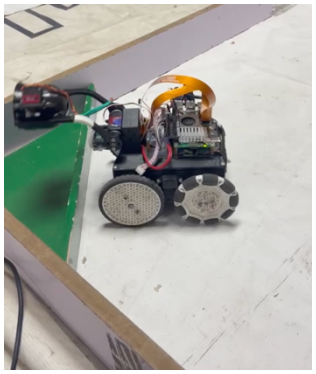
Team Photo

2. Project Planning

a. Overall Project Plan

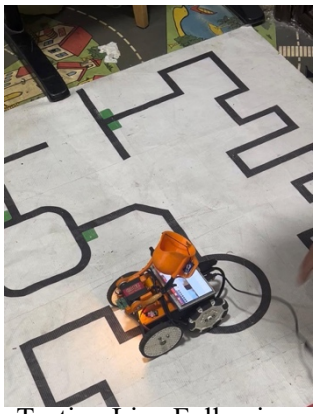
Our primary objective was to build a highly reliable robot capable of seamlessly transitioning between precise line-following and autonomous search-and-rescue within the evacuation zone. Based on competition constraints, we defined strict requirements:

1. *Physical:* Must traverse speed bumps without bottoming out or losing power.
2. *Computational:* Must process complex AI vision at high frame rates (30+ FPS) without thermal throttling.
3. *Electrical:* Must guarantee clean power to logic controllers even during high-torque motor stalls.

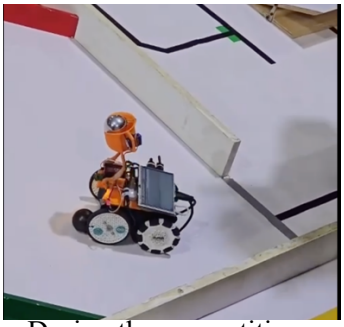

Robot detecting evacuation

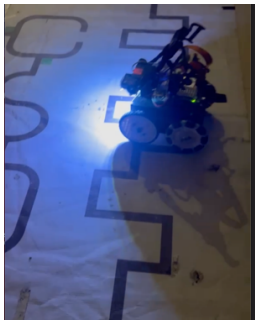
Milestones & Timeline: We utilized 9-month development cycle post-regionals, utilizing weekly progress gates to review milestones.

- *Milestone 1 (Month 1-2): Mechanical Prototyping (Youssef).* Requirement: Redesign chassis for bump clearance. Gate: Physical obstacle traversal test.
- *Milestone 2 (Month 3-4): Electrical Isolation (Ruba).* Requirement: Design and order custom PCB. Gate: Continuity and load testing of the bare board.
- *Milestone 3 (Month 5-6): Software Architecture (Maya & Diala).* Requirement: Establish stable UART bridge between Pi 5 and Nano. Gate: Successfully passing simulated motor commands based on dummy vision data.
- *Milestone 4 (Month 7-8): AI Training (Maya).* Requirement: Train YOLOv8 model on Apple Silicon (MPS). Gate: Achieve >0.80 F1-confidence score on validation data.
- *Milestone 5 (Month 9): Full Integration & Tuning (All).* Requirement: Autonomous competition runs**.**


Testing Line Following


Updated Robot Version 2

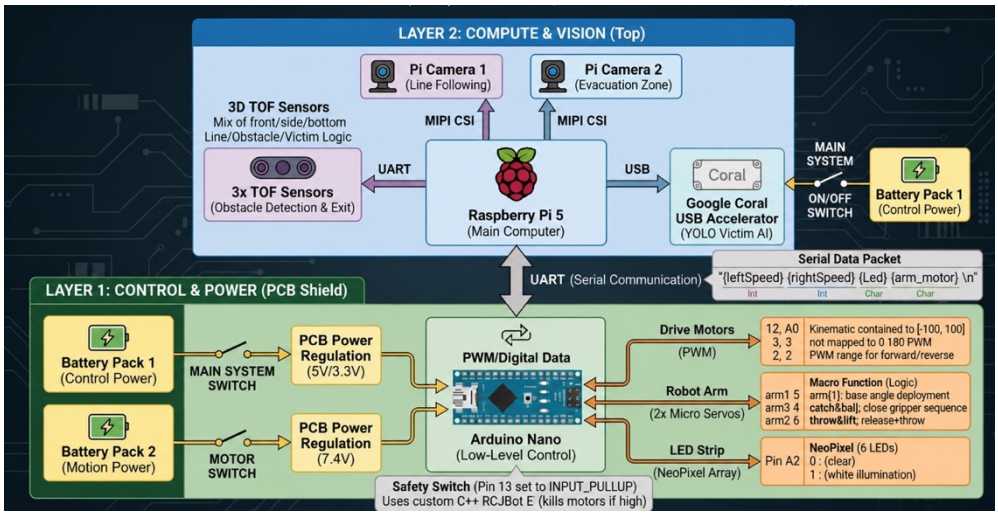

During the competition


Testing Light conditions

b. Integration Plan

Our integration plan was structured around a dual-layer "Brain and Brawn" architecture.

- Layer 2 (Compute & Vision): The Raspberry Pi 5 gathers data from dual MIPI-CSI cameras and three ToF sensors (UART). It processes this data and sends simple, single-byte action packets over a dedicated UART bridge.
- Layer 1 (Control & Power): The Arduino Nano receives these packets and translates them into physical actuation via the custom PCB, which routes PWM signals to the motors and servos.



SID Diagram

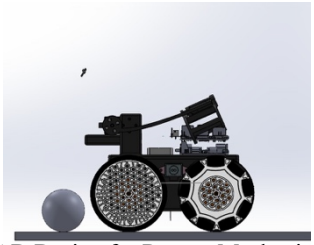
3. Hardware

Our hardware design prioritizes reliability and serviceability, utilizing modular sub-assemblies connected via a central custom PCB.

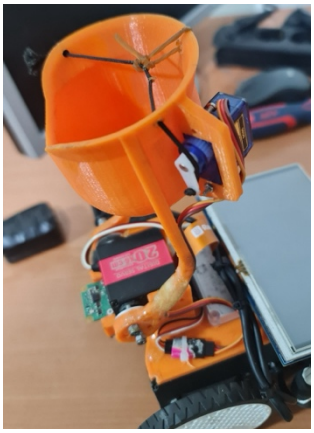
CAD Design using SolidWorks

a. Mechanical Design and Manufacturing

- **Main Structure:** The chassis was custom-designed in SolidWorks and 3D printed. A major iteration from our regional robot was significantly increasing the ground clearance to prevent high-centering on standard RCJ speed bumps.
- **Actuators and Power Train:** We utilize four REV Smart Servos equipped with metal gearing to prevent stripping under heavy loads, driving REV DUO motion wheels. These wheels were specifically chosen for their high traction coefficient on standard tile/banner material.
- **Rescue Mechanism:** The victim catching mechanism is an innovative dual-micro-servo sweeping arm. It is designed to be highly compact during line following but expand rapidly upon entering the evacuation zone to secure victims against a backplate.
- **Innovative Solution (Battery Holder):** We discovered that traversing speed bumps caused micro-disconnections in standard battery sleds. To solve this, Youssef designed a custom 3D-printed enclosure that rigidly secures the Panasonic NCR18650 cells, eliminating power loss during violent maneuvers.



CAD Design for Rescue Mechanism



Servo Motors representation

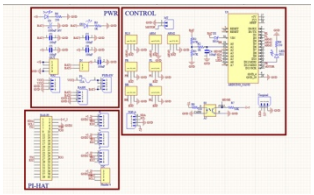
b. Electronic Design and Manufacturing

 Main Controller & PCB: The heart of our electronics is a custom-designed PCB engineered in Altium Designer. This board houses our Arduino Nano, centralizes all sensor headers, and reduces loose wiring by 60%.

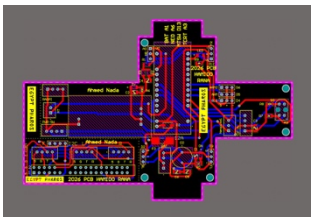
 Sensors: We use a dual-camera system to eliminate mechanical interference. Camera 1 is angled strictly for line tracking, while Camera 2 is a wide-angle lens dedicated to the evacuation zone. We also utilize 3x Yahboom TinyF ToF sensors via UART for precise wall-tracking and an LCD touchscreen for headless debugging.

 Power Subsystem: To ensure reliability, we separated the power supplies. Battery Pack 1 (7.4V) powers the logic (Pi 5, Nano, Sensors), while Battery Pack 2 (7.4V) is isolated solely for the drive motors.

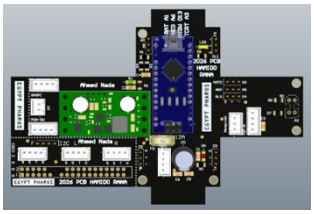
 Testing & Quality Assurance: Our primary reliability test involved intentionally stalling the drive wheels and monitoring the logic voltage using an oscilloscope. The custom PCB power isolation ensured the Pi 5 never experienced a brownout during stalls.



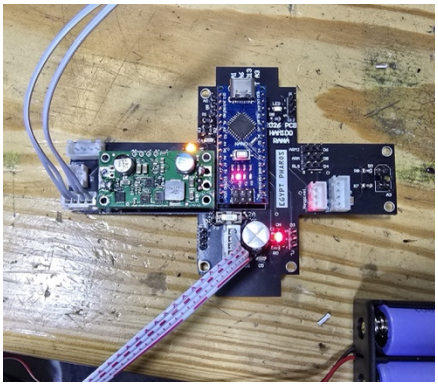
PCB Schematic using Altium



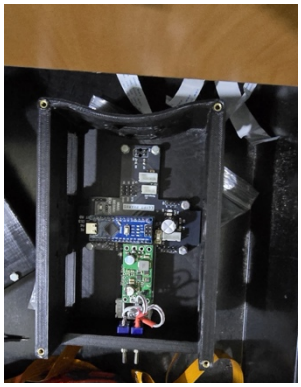
PCB Design using Altium



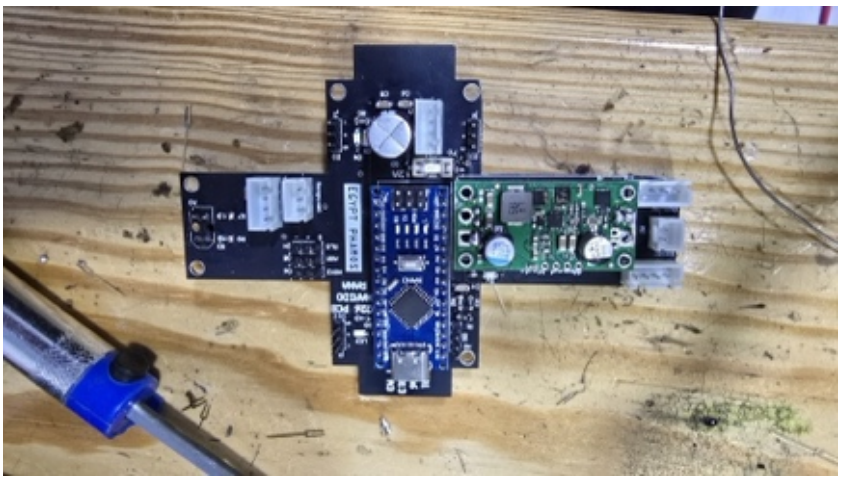
PCB 3D using Altium



PCB Testing



PCB fitting inside the Robot



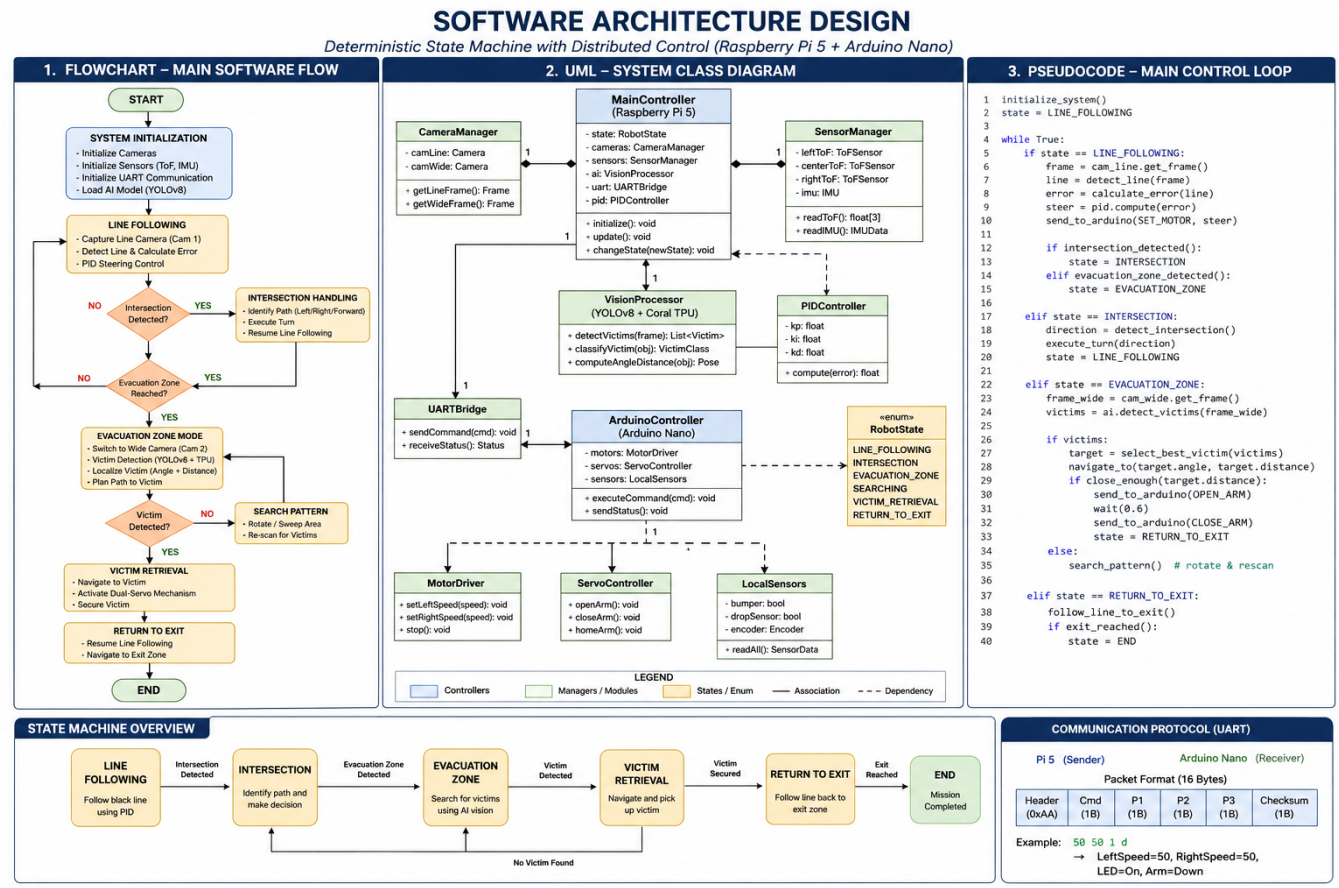
PCB final

4. Software

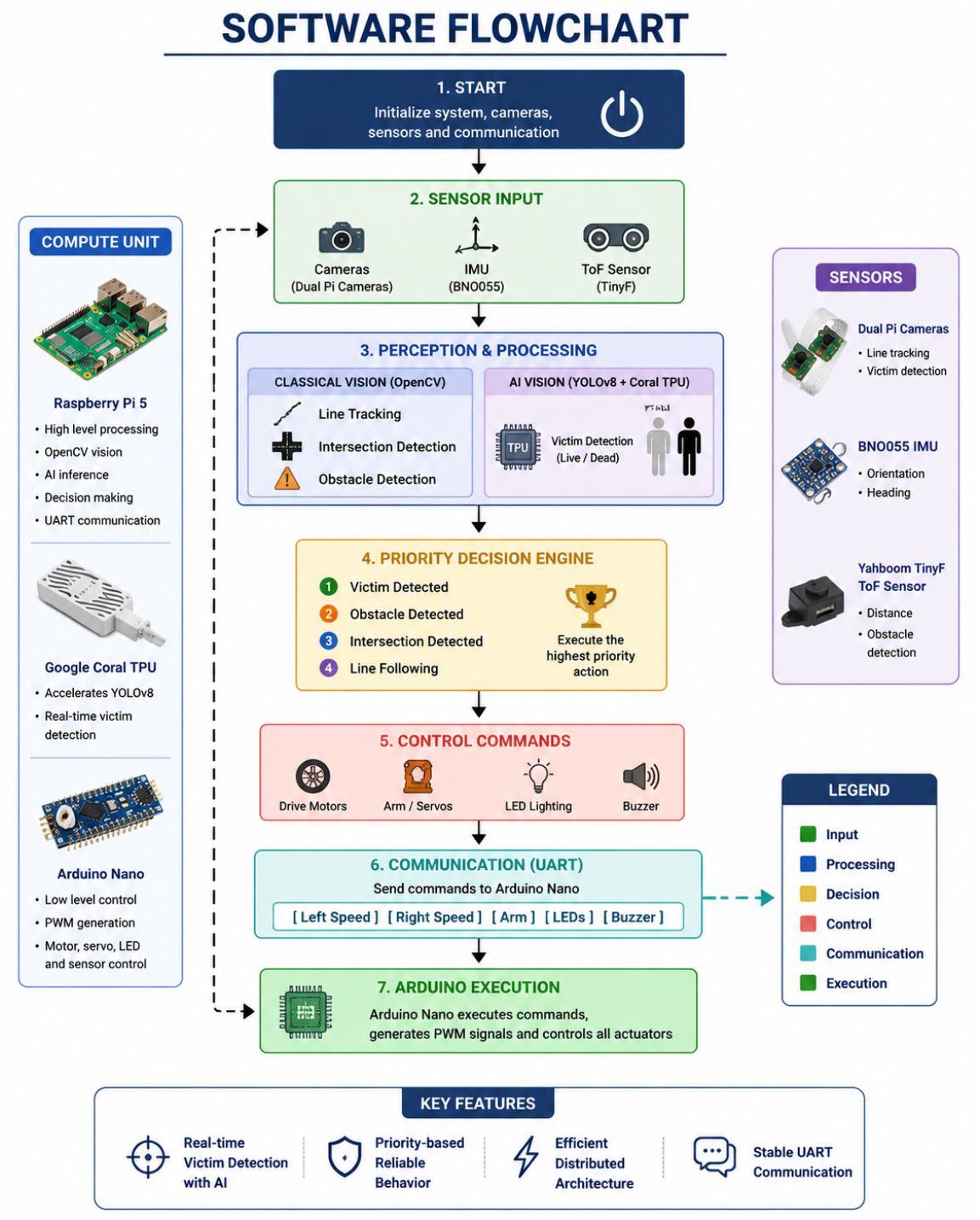
a. General software architecture

Our software employs a deterministic state machine divided between high-level perception and low-level control.

- **Raspberry Pi (Python):** Captures frames at 90 FPS. The main_code.py script slices the frame into specific Regions of Interest (ROIs) for the black line, green intersections, and red obstacles. It calculates positional error using OpenCV contours and computes the PID steering value.
- **Arduino Nano (C++):** Runs our custom library. It receives the packaged UART string (e.g., 50 50 1 d \n -> LeftSpeed, RightSpeed, LED State, Arm State) and executes it with strict deterministic timing.



The system operates as a deterministic state machine. Sensor data are processed by the Raspberry Pi, while the Arduino executes real-time commands. State transitions are triggered by line features, obstacle detection, victim detection, and evacuation zone events.



b. Innovative solutions

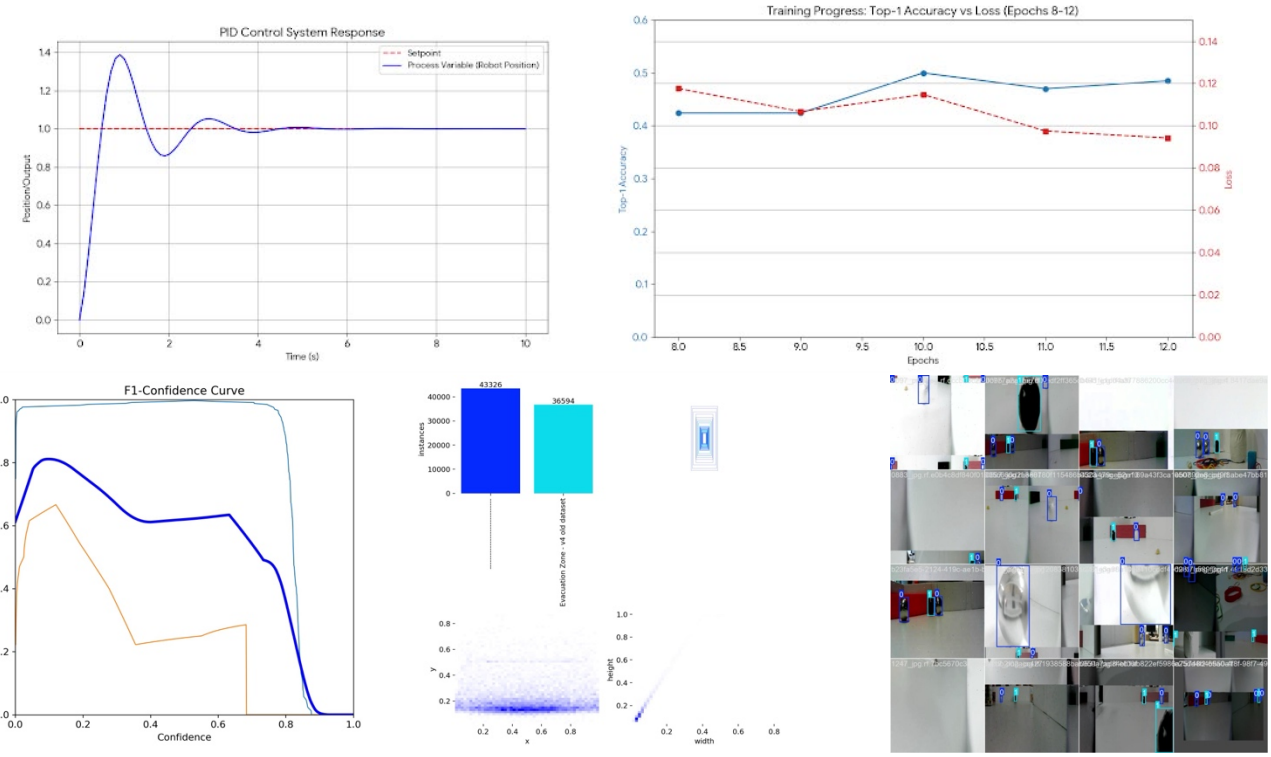
- AI-Accelerated Victim Detection: Instead of relying on unreliable color-blob tracking for victims, we custom-trained a YOLOv8 object detection model over 43,000 images to recognize live (silver) and dead (black) victims under varying lighting. To prevent the Raspberry Pi CPU from bottlenecking, we offloaded the model inference to a Google Coral USB Accelerator TPU.

5. Performance evaluation

 Reliability Testing: We conducted extensive real-world testing on modular track segments. Our PID line-following error margin was logged and graphed over time, allowing us to dial in our Kp, Ki , and Kd values until oscillations were eliminated. Our YOLOv8 model was validated with an F1-Confidence peak score of 0.81.

 Insightful Problem Evaluation: During early testing, we discovered a major integration issue: The Raspberry Pi would occasionally reboot when the robot attempted to push a heavy obstacle, causing a total system failure. Through quality assurance testing, Ruba identified that the motor stall current was causing a severe voltage drop across the shared power rail.

 The Fix: This directly informed our decision to abandon a shared power system. We redesigned the electrical architecture to include completely isolated 7.4V battery packs for logic and motion, routed through our custom PCB. Subsequent tests showed 100% uptime regardless of motor load.



6. Conclusion

The Egypt Pharaoh team successfully engineered a RoboCup Junior Rescue Line robot that prioritizes electrical stability, mechanical robustness, and advanced AI perception. By moving away from off-the-shelf kits and developing a custom PCB, a distributed computing architecture, and a TPU-accelerated YOLOv8 vision system, we created a highly competitive platform. Our rigorous milestone-based planning and isolated power solutions directly addressed the reliability challenges of the competition, ensuring our success at the regional level and preparing us for the World Finals.